Guía de Actividades
PrácticoExperimentales Nro. 001

1. Datos Generales

Asignatura	Estructura de Datos
Integrantes	Julián Merino Matías Encarnación Matías Lavanda Sherman Abarca Jefferson Sarango
Ciclo	3ro A / B
Unidad	1: Estructuras de datos estáticas y dinámicas
Resultado de aprendizaje de la unidad	Modela el uso de memoria y diseña Tipos de Datos Abstractos (TDA) para comprender las diferencias de complejidad espacial, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	01
Título de la Práctica	Micro-TDAs, Abstracción de Datos y Análisis de Memoria en Motores V8
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 01 y viernes 02 de octubre 2026
Horario	Jueves 01 de octubre 10:30 – 11:30 Viernes 02 de octubre 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

- Modelar Tipos de Datos Abstractos (TDA) simples utilizando clases y constructores en JavaScript moderno.
- Medir empíricamente el impacto en la memoria Heap (complejidad espacial S(n) al procesar estructuras de datos basadas en tipos primitivos frente a arreglos de objetos masivos.
- Evaluar mediante el análisis del recolector de basura (Garbage Collector) las decisiones de diseño físico y su impacto en la sostenibilidad computacional (Green Computing).

3. Materiales y Reactivos

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación técnica oficial (ABI).
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Procedimiento / Metodología

Enfoque PID-2025:

- ABP (Aprendizaje Basado en Problemas): El equipo es contratado por un proyecto de geolocalización satelital sostenible. Deben cargar $N = 10^6$ coordenadas en memoria. El problema radica en que instanciar 10^6 objetos colapsa los límites del entorno. Se debe diseñar y medir la alternativa más eficiente.
- ABI (Aprendizaje Basado en Investigación): Los estudiantes deberán investigar cómo el motor de JavaScript (V8) reserva memoria contigua vs. memoria dinámica.
- Modalidad: Individual y discusión grupal posterior.

Tarea 1: Preparación del Entorno de Medición (Profiler)

Para medir la huella de memoria en JavaScript, utilizaremos el módulo integrado process.

1. Cree un nuevo directorio para el proyecto y abra su editor de texto.
2. Cree un archivo denominado profiler.js.
3. Implemente la función de medición de memoria RAM (Heap):

```
/**
* Tarea 1: Utilidad para medir la memoria Heap en Megabytes (MB).
* El Heap es el espacio de memoria dinámico donde V8 almacena objetos y variables.
 */
function getMemoryUsage() {
    const memoryData = process.memoryUsage();
    // Convertimos de bytes a Megabytes (MB) usando notación matemática estándar
    const heapUsedMB = Math.round(memoryData.heapUsed / 1024 / 1024 * 100)
/ 100;
    return heapUsedMB;
}

module.exports = { getMemoryUsage };
```

Tarea 2: Enfoque 1 - Modelado basado en Arreglo de Objetos (TDA) En esta tarea, crearemos N objetos. Esta es la representación de más alto nivel de abstracción.

1. Cree un archivo llamado lab01_objetos.js.
2. Incluya el siguiente código, el cual creará $1,000,000$ de instancias de coordenadas.

```
const { getMemoryUsage } = require('./profiler');

const N = 1000000; // 1 millón de registros const
memoriaInicial = getMemoryUsage();

// TDA: Coordenada basada en Objetos class
CoordenadaObj {          constructor(lat, lng) {
                this.lat = lat; // Número de punto flotante de 64 bits
this.lng = lng;
        }
}

// Almacenamos en memoria let
coordenadas = [];
for (let i = 0; i < N; i++) {
        coordenadas.push(new CoordenadaObj(i * 0.1, i * -0.1));
}

const memoriaFinal = getMemoryUsage(); console.log(`[Enfoque Objetos]
Memoria inicial: ${memoriaInicial} MB`); console.log(`[Enfoque Objetos] Memoria
final: ${memoriaFinal} MB`);
console.log(`[Enfoque Objetos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);
```

3. Ejecute en su terminal: node lab01_objetos.js y anote el resultado neto.

Tarea 3: Enfoque 2 - Modelado basado en Arreglos de Primitivos paralelos (Data-Oriented Design)

1. Cree un archivo llamado lab01_primitivos.js.
2. Implemente la estructura de datos desacoplando la abstracción del objeto a arreglos paralelos unidimensionales fuertemente tipados (Float64Array). const { getMemoryUsage } = require('./profiler'); const N = 1000000;

```
const memoriaInicial = getMemoryUsage();

// TDA Desacoplado: Uso de Typed Arrays para obligar a V8 a usar memoria contigua
```

```
// La capacidad total requerida es N elementos de 8 bytes (64 bits) cada uno let lat =
new Float64Array(N); let lng = new Float64Array(N);

for (let i = 0; i < N; i++) {
lat[i] = i * 0.1;          lng[i] =
i * -0.1;
}

const memoriaFinal = getMemoryUsage(); console.log(`[Enfoque Primitivos]
Memoria inicial: ${memoriaInicial} MB`); console.log(`[Enfoque Primitivos] Memoria
final: ${memoriaFinal} MB`);
console.log(`[Enfoque Primitivos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);
```

3. Ejecute en su terminal: node lab01_primitivos.js y anote el resultado.

Tarea 4: Investigación y Cálculo de Complejidad

Basado en los resultados, el estudiante debe aplicar ABI para investigar el porqué de la diferencia matemática.

1. **Busque en la documentación oficial de V8 cómo se empaquetan las propiedades en los Hidden Classes de JavaScript.**

   Las propiedades en los Hidden Classes de JavaScript se organizan mediante tres esquemas de almacenamiento:

   In Object Properties: Son almacenadas directamente dentro del bloque de memoria contiguo asignado al propio JSObject, inmediatamente después de su cabecera. V8 calcula cuántas propiedades tendrá probablemente el objeto durante la instanciación y reserva estos slots internamente.

   Fast Properties: Si el objeto supera la cantidad de espacios in-object prereservados, las propiedades adicionales se mueven a un arreglo contiguo auxiliar (FixedArray) apuntado por el puntero properties.

   Slow Properties: Si las propiedades se añaden o eliminan de forma dinámica e impredecible, V8 degrada el almacenamiento a una tabla de hash (NameDictionary), sacrificando el acceso por desplazamiento (*offset*).

5. Resultados esperados:

- El estudiante obtendrá evidencia empírica en consola demostrando que el manejo de Tipos de Datos Abstractos complejos mediante instanciación masiva de objetos genera un overhead significativo en la memoria (heap), incrementando la latencia del recolector de basura. Se espera una diferencia de consumo sustancial a favor de los Typed Arrays (`Float64Array`).

Implementación del código en lab01_objetos.js (Enfoque 1)

AA(Semana1) > JS lab01_objetos.js > ...

```
const { getMemoryUsage } = require('./profiler');
const N = 1000000; // 1 millón de registros
const memoriaInicial = getMemoryUsage();
// TDA: Coordenada basada en Objetos
class CoordenadaObj {     constructor(lat, lng) {
        this.lat = lat; // Número de punto flotante de 64 bits
        this.lng = lng;
    }
}
// Almacenamos en memoria
let coordenadas = [];
for (let i = 0; i < N; i++) {
    coordenadas.push(new CoordenadaObj(i * 0.1, i * -0.1));
}
const memoriaFinal = getMemoryUsage();
console.log(`[Enfoque Objetos] Memoria inicial: ${memoriaInicial} MB`);
console.log(`[Enfoque Objetos] Memoria final: ${memoriaFinal} MB`);
console.log(`[Enfoque Objetos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);

```

Implementación del código en lab01_primitivos.js (Enfoque 2)

AA(Semana1) > JS lab01_primitivos.js > ...

```
const { getMemoryUsage } = require('./profiler'); const N = 1000000;
const memoriaInicial = getMemoryUsage();
// TDA Desacoplado: Uso de Typed Arrays para obligar a V8 a usar memoria contigua
// La capacidad total requerida es N elementos de 8 bytes (64 bits) cada uno
let lat = new Float64Array(N); let lng = new Float64Array(N);
for (let i = 0; i < N; i++) {     lat[i] = i * 0.1;     lng[i] = i * -0.1;
}
const memoriaFinal = getMemoryUsage();
console.log(`[Enfoque Primitivos] Memoria inicial: ${memoriaInicial} MB`);
console.log(`[Enfoque Primitivos] Memoria final: ${memoriaFinal} MB`);
console.log(`[Enfoque Primitivos] Consumo Neto: ${memoriaFinal - memoriaInicial} MB`);

```

Resultados en lab01_objetos.js (Enfoque 1)

```
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos> node "c:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos\AA(Semana1)\lab01_objetos.js"
[Enfoque Objetos] Memoria inicial: 3.75 MB
[Enfoque Objetos] Memoria final: 94.05 MB
[Enfoque Objetos] Consumo Neto: 90.3 MB
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos>
```

Resultados en lab01_primitivos.js (Enfoque 2)

```
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos> node "c:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos\AA(Semana1)\lab01_primitivos.js"
[Enfoque Primitivos] Memoria inicial: 3.75 MB
[Enfoque Primitivos] Memoria final: 4.09 MB
[Enfoque Primitivos] Consumo Neto: 0.33999999999999986 MB
PS C:\Users\nicof\OneDrive\Desktop\Archivos\Estructura de Datos>
```

6. Preguntas de Control:

 Responda las siguientes preguntas reflexionando sobre las causas fundamentales de los fenómenos computacionales observados:

1. **Causa Material: ¿De qué está constituido fundamentalmente un objeto en la memoria del entorno de ejecución (V8), y en qué difiere la "materia" de un objeto de la de un valor primitivo continuo en un `Float64Array`?**

   Un objeto en memoria (JSObject) está constituido por una cabecera de metadatos para la recolección de basura (HeapObject header), un puntero obligatorio a su Map (la clase oculta o Hidden Class que define la estructura del objeto y sus offsets), y referencias a contenedores auxiliares para sus elementos indexados (Elements) y propiedades externas (Properties), sumado a las propiedades integradas en el objeto (In-object properties).

   La materia de un valor primitivo en un Float64Array consiste exclusivamente en una secuencia binaria contigua de 64 bits (8 bytes) en formato de punto flotante puro según el estándar IEEE 754, alojada dentro de un bloque de memoria contiguo (ArrayBuffer). Difiere radicalmente de la materia de un objeto en que carece por completo de cabeceras individuales, punteros a clases ocultas (Map), envoltorios (wrappers) o sobrecoste directo para el recolector de basura por cada elemento. Esta diferencia física en la memoria explica los resultados de las pruebas ejecutadas: la creación de un millón de objetos generó un consumo neto de 90.3 MB debido a toda la sobrecarga estructural de metadatos por cada instancia, mientras que el almacenamiento directo de primitivos en Float64Array redujo el consumo neto a aproximadamente 0.34 MB al reservar únicamente el espacio contiguo estricto para los bytes numéricos.

2. **Causa Formal: ¿En qué medida la "forma" o estructura lógica que imponemos a un Tipo de Dato Abstracto (TDA) define y restringe los límites de su eficiencia física en el hardware?**

   La estructura lógica de un TDA restringe la eficiencia física del hardware al dictar la disposición de los datos en memoria (memory layout) y el aprovechamiento de la caché de la CPU. Un diseño orientado a objetos como CoordenadaObj fragmenta la memoria e introduce metadatos que destruyen la localidad espacial y causan cache misses, elevando el sobrecoste a 90.3 MB. Por el contrario, un diseño desacoplado en primitivos contiguos como Float64Array coincide con la arquitectura física del procesador, llenando

las líneas de caché con datos útiles y reduciendo el consumo estrictamente a los 0.34 MB de valores numéricos indispensables.

2. **Causa Eficiente: Al observar el consumo de memoria, ¿es el intérprete de JavaScript el verdadero artífice (causa eficiente) del desperdicio de recursos, o reside la responsabilidad primaria en la decisión arquitectónica del desarrollador?**

   La responsabilidad reside en la decisión arquitectónica del desarrollador, al crear objetos se crean junto a ellos cabeceras, mapas de forma y punteros para cada entidad, los objetos son mas pesados y por esta razón en las pruebas se tuvo un resultado de consumo neto de 90.3 MB, por otra parte, usando el Float64Array se asigna un bloque de memoria sin sobrecosto reduciendo el consumo de memoria neto a 0.34 MB.

3. **Causa Final: ¿Cuál es el propósito último de abstraer los datos en la programación orientada a objetos si, en su esencia más fundamental, toda optimización crítica requiere retornar a la disposición contigua de bits y bytes en la memoria?**

   El propósito de abstraer datos mediante la programación orientada a objetos es optimizar la cognición humana y la mantenibilidad del software, permitiendo a los desarrolladores modelar dominios del mundo real, encapsular comportamiento y escalar sistemas a través de conceptos intuitivos. La abstracción no fue concebida para agradar al hardware, sino para reducir la carga mental del programador, favorecer el trabajo en equipo y prevenir errores que surgen al manipular memoria de bajo nivel. Aunque retornar a la disposición contigua de bits mediante Float64Array maximiza la eficiencia física reduciendo el consumo de manera drástica de 90.3 MB a solo 0.34 MB, aplicar esta optimización a todo el sistema destruiría la expresividad y flexibilidad de la arquitectura.

4. **Esencia y Existencia: Si dos estructuras de datos (como el Enfoque 1 y el Enfoque 2) resuelven exactamente el mismo problema lógico arrojando los mismos resultados, pero existen físicamente ocupando volúmenes de energía y memoria drásticamente distintos, ¿son en esencia la misma solución? Argumente.**

   No son la misma solución, la naturaleza de un desarrollo en informática abarca tanto su equivalencia lógica como su comportamiento físico en el hardware. Mientras que el Enfoque 1 prioriza la abstracción del desarrollador a costa de un elevado costo material con 90.3 MB, el Enfoque 2 se enfoca en la eficiencia mecánica utilizando apenas 0.34 MB. En consecuencia, son soluciones lógicamente equivalentes, pero físicamente distintas, ya que la huella en memoria, energía y uso de la CPU forma parte de la esencia misma de cualquier software en el mundo real.

7. Conclusiones

- Utilizar objetos tradicionales como `CoordenadaObj` facilita la comprensión y el trabajo del programador, pero genera un gran desperdicio de memoria (90.3 MB) debido a la estructura extra que requieren internamente. Por otro lado, usar estructuras más crudas y directas como `Float64Array` organiza los datos sin sobrecostes, reduciendo el consumo en el *heap* a tan solo 0.34 MB. Es decir, la abstracción ayuda al humano, pero le cuesta más a la máquina.
- Aunque los dos enfoques realizan exactamente la misma tarea matemática iterando un millón de veces, físicamente son soluciones completamente distintas. La función de medición getMemoryUsage() deja en evidencia esta enorme diferencia en el consumo de recursos. Esto demuestra que es el desarrollador quien tiene la responsabilidad de decidir cuándo usar un código amigable y cuándo aplicar una optimización extrema para no saturar el hardware.

8. Link del Repositorio

https://github.com/Jullxn28/Ciclo3_EstructuraDeDatos_Practica.git

Evaluación

Criterio Evaluado	Puntaje Máximo	Evidencia Esperada
1. Implementación TDA y Entorno	2 puntos	Capturas de pantalla con la ejecución correcta de los archivos .js en consola sin errores de sintaxis.
2. Medición y Tabulación de Datos	2 puntos	Tabla comparativa de consumos en MB entregada en el informe que demuestre las diferencias entre objetos y primitivos.
3. Análisis Matemático / Teórico	3 puntos	Explicación fundamentada del

(ABI)		overhead utilizando notación Big-O e investigación sobre Hidden Classes en V8.
4. Respuestas Aristotélicas	3 puntos	Respuestas a las preguntas de control que demuestren profundidad de análisis sobre la naturaleza lógica y física de los datos.
Total	10 puntos	

7. Bibliografía

1. P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023. [Online]. Available: https://open.palni.org/
2. Mozilla Developer Network, "Memory management", MDN Web Docs, 2023. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Ma nagement
3. O. Levin, Discrete Mathematics: An Open Introduction, Autoeditado, 2024.

8. Elaboración y Aprobación

- Elaborado por: nombre del Docente responsable de la práctica.
- Aprobado por: nombre del Director de Carrera.

Elaborado por	Cristian R. Narváez Guillén Docente	
Aprobado por	Edison L Coronel Romero Director de Carrera	